OPERATING SYSTEM (CT 656)

Chapter 2: Process Management

Tribhuvan University | BCT | Year III / Part II

Full Marks: 80 | Pass Marks: 32 | Time: 3 hrs.

Q1. Define process. Describe the various states of a process. How significant is process hierarchy? [1+2+2]

Answer:

A process is a program in execution. It is more than just the program code — it includes the current activity (program counter, registers), the process stack (temporary data), the data section (global variables), and a heap (dynamically allocated memory).

States of a Process (5-State Model):

- New: The process is being created. It has been submitted but not yet admitted to the ready queue.
- Ready: The process is waiting to be assigned to the CPU. It has all resources except the CPU.
- Running: The process is currently being executed by the CPU. Only one process per CPU core can be in this state at a time.
- Waiting (Blocked): The process is waiting for some event to occur (e.g., I/O completion, a signal). It cannot proceed until the event happens.
- Terminated: The process has finished execution. The OS reclaims its resources.

State Transitions:

- New → Ready: Process admitted to ready queue
- Ready → Running: Scheduler dispatches the process to CPU
- Running → Ready: Process preempted (time quantum expired or higher priority process arrives)
- Running → Waiting: Process waits for I/O or event
- Waiting → Ready: I/O or event completes
- Running → Terminated: Process finishes or is killed

7-State Model adds two more states:

- Suspend Ready: Process moved to secondary storage (swapped out) from ready state
- Suspend Wait: Process moved to secondary storage while waiting

Significance of Process Hierarchy:

- When a process creates another process using fork(), a parent-child relationship is formed, creating a process tree (hierarchy)
- A parent can wait for its child to complete using wait() system call
- Resources can be shared between parent and child (files, memory)
- The hierarchy allows organized process management — if a parent is killed, all its children can be systematically terminated

- init process (PID=1 in Linux) is the root of the process hierarchy; all other processes descend from it

Q2. Difference between process and thread. Explain how multi-threading provides better solution than single threading solution. [1+3]

Answer:

Process	Thread
An independent program in execution with its own memory space	A lightweight unit of execution within a process, sharing process memory
Has its own address space, code, data, heap, stack	Shares code, data, heap with other threads; has its own stack and registers
Process creation is expensive (new memory space required)	Thread creation is cheap (shares existing process resources)
Inter-process communication (IPC) is complex and slow	Threads communicate easily via shared memory (but need synchronization)
Context switch between processes is slow	Context switch between threads is faster
If one process crashes, others are not affected	If one thread crashes, it can crash the entire process
Example: Running two separate programs	Example: Multiple tabs in a browser (threads of one process)

Multi-threading vs Single-threading:

Single Threading	Multi-threading
One thread per process; tasks execute sequentially	Multiple threads per process; tasks execute concurrently
If one task blocks (e.g., I/O), entire program waits	While one thread waits for I/O, other threads continue executing
Poor CPU utilization on multi-core systems	Threads can run on different cores — true parallelism on multi-core CPUs
Simple to program, no synchronization needed	More complex — requires synchronization (mutex, semaphore) to prevent race conditions
Example: Single-threaded web server handles one request at a time	Example: Multi-threaded web server handles many requests simultaneously

Advantages of Multi-threading:

- Responsiveness: GUI applications remain responsive — one thread handles UI, another does computation
- Resource Sharing: Threads share memory, reducing overhead compared to separate processes
- Economy: Creating/managing threads is cheaper than processes

- Scalability: On multi-processor systems, threads execute in parallel, improving performance

---

Q3. What is the rationale behind defining the quantum for Round Robin scheduling? Explain. Make a schedule for the processes as per Round Robin scheduling algorithm (quantum size = 4 ms). Calculate average turnaround time, average waiting time, CPU utilization and throughput. [3+5] Process: A(BT=3, AT=0), B(BT=7, AT=1), C(BT=2, AT=2), D(BT=5, AT=3), E(BT=1, AT=4)

Answer:

Rationale for Defining Quantum in Round Robin:

- The quantum (time slice) determines how long each process runs before being preempted
- If quantum is too small: Too many context switches → high overhead, poor CPU efficiency
- If quantum is too large: Behaves like FCFS → poor response time for short processes
- Ideal quantum: Large enough to minimize context switching, small enough to give good response time (typically 10-100 ms)
- Rule of thumb: 80% of CPU bursts should be shorter than the quantum

Round Robin Scheduling (Quantum = 4 ms):

Processes: A(3,0), B(7,1), C(2,2), D(5,3), E(1,4)

Gantt Chart Execution (Queue management):

Process	AT	BT	Priority/Queue Order
A	0	3	Arrives first
B	1	7	Arrives at t=1
C	2	2	Arrives at t=2
D	3	5	Arrives at t=3
E	4	1	Arrives at t=4

Execution sequence with Q=4:

- t=0-3: A runs (BT=3, completes) → A finishes at t=3
- t=3-7: B runs for 4 units (remaining BT=3)
- t=7-9: C runs for 2 units (completes)
- t=9-13: D runs for 4 units (remaining BT=1)
- t=13-14: E runs for 1 unit (completes)
- t=14-17: B runs for 3 units (completes)
- t=17-18: D runs for 1 unit (completes)

Calculations:

Process	AT	BT	CT	TAT = CT-AT	WT = TAT-BT	Notes
A	0	3	3	3	0	Completes first
B	1	7	17	16	9	Preempted once
C	2	2	9	7	5	Completes 2nd run

Process	AT	BT	CT	TAT = CT-AT	WT = TAT-BT	Notes
D	3	5	18	15	10	Preempted once
E	4	1	14	10	9	Short burst

Average TAT: (3 + 16 + 7 + 15 + 10) / 5 = 51 / 5 = 10.2 ms
Average WT: (0 + 9 + 5 + 10 + 9) / 5 = 33 / 5 = 6.6 ms
CPU Utilization: Total Burst Time / Total Time = 18/18 × 100% = 100%
Throughput: 5 processes / 18 ms = 0.278 processes/ms

Q4. Schedule the following processes according to SRTN and Preemptive Priority Algorithm to calculate average TAT and WT. [7] P0(AT=0,BT=8,Pri=2), P1(AT=2,BT=2,Pri=4), P2(AT=4,BT=12,Pri=0), P3(AT=8,BT=4,Pri=3), P4(AT=12,BT=10,Pri=1) [Lowest value = Highest priority]

Answer:

SRTN (Shortest Remaining Time Next) — Preemptive SJF:
At every new process arrival, compare remaining burst times and run the shortest.

SRTN Execution:

- t=0: Only P0 available → P0 runs. Remaining: P0=8
- t=2: P1 arrives (BT=2). P1 < P0(rem=6) → P1 preempts P0. Run P1.
- t=4: P1 completes (CT=4). P2 arrives (BT=12). P0(rem=6) < P2(12) → P0 runs.
- t=8: P3 arrives (BT=4). P0(rem=2) < P3(4) → P0 continues.
- t=10: P0 completes (CT=10). P3(4) < P2(12) → P3 runs.
- t=12: P4 arrives (BT=10). P3(rem=2) < P4(10) → P3 continues.
- t=14: P3 completes (CT=14). P4(10) < P2(12) → P4 runs.
- t=24: P4 completes (CT=24). P2 runs.
- t=36: P2 completes (CT=36).

Process	AT	BT	Pri	CT	TAT=CT-AT	WT=TAT-BT	Notes
P0	0	8	2	10	10	2	SRTN
P1	2	2	4	4	2	0	SRTN
P2	4	12	0	36	32	20	SRTN
P3	8	4	3	14	6	2	SRTN
P4	12	10	1	24	12	2	SRTN

SRTN Average TAT: (10+2+32+6+12)/5 = 62/5 = 12.4 ms
SRTN Average WT: (2+0+20+2+2)/5 = 26/5 = 5.2 ms

Preemptive Priority Scheduling (Lower number = Higher priority):

- t=0: P0(pri=2) runs.

- t=2: P1(pri=4) arrives. P0(2) has higher priority → P0 continues.
- t=4: P2(pri=0) arrives. P2 has highest priority → P2 preempts P0.
- t=8: P3(pri=3) arrives. P2(0) still highest → P2 continues.
- t=12: P4(pri=1) arrives. P2(0) still highest → P2 continues.
- t=16: P2 completes. P4(1) next highest → P4 runs.
- t=26: P4 completes. P0(rem=6, pri=2) → P0 runs.
- t=32: P0 completes. P3(pri=3) → P3 runs.
- t=36: P3 completes. P1(pri=4) → P1 runs.
- t=38: P1 completes.

Process	AT	BT	Pri	CT	TAT=CT-AT	WT=TAT-BT	Notes
P0	0	8	2	32	32	24	Priority
P1	2	2	4	38	36	34	Priority
P2	4	12	0	16	12	0	Priority
P3	8	4	3	36	28	24	Priority
P4	12	10	1	26	14	4	Priority

Priority Average TAT: (32+36+12+28+14)/5 = 122/5 = 24.4 ms

Priority Average WT: (24+34+0+24+4)/5 = 86/5 = 17.2 ms

Q5. Schedule the following set of processes according to HRRN and Round-Robin scheduling algorithm with Quantum time = 3 ms and calculate the average waiting time and average turn-around time. [8] A(AT=0,CPU=12), B(AT=2,CPU=8), C(AT=5,CPU=7), D(AT=10,CPU=9)

Answer:

HRRN (Highest Response Ratio Next):

Response Ratio = (Waiting Time + Burst Time) / Burst Time

At each scheduling decision, compute response ratio for all ready processes and select the one with highest ratio.

HRRN Execution:

- t=0: Only A ready → A runs for 12 ms. CT(A)=12.
- t=12: B(WT=10,BT=8, RR=(10+8)/8=2.25), C(WT=7,BT=7, RR=(7+7)/7=2.0), D(WT=2,BT=9, RR=(2+9)/9=1.22) → B wins → B runs for 8 ms. CT(B)=20.
- t=20: C(WT=15,BT=7, RR=(15+7)/7=3.14), D(WT=10,BT=9, RR=(10+9)/9=2.11) → C wins → C runs for 7 ms. CT(C)=27.
- t=27: D runs → CT(D)=36.

Process	AT	BT	CT	TAT=CT-AT	WT=TAT-BT	RR at Select
A	0	12	12	12	0	1.0 (only one)
B	2	8	20	18	10	2.25

Process	AT	BT	CT	TAT=CT-AT	WT=TAT-BT	RR at Select
C	5	7	27	22	15	3.14
D	10	9	36	26	17	2.11

HRRN Average TAT: (12+18+22+26)/4 = 78/4 = 19.5 ms
HRRN Average WT: (0+10+15+17)/4 = 42/4 = 10.5 ms

Round Robin (Q=3):

- t=0-3: A(rem=9)
- t=3-6: B(rem=5) [B arrived at 2]
- t=6-9: C(rem=4) [C arrived at 5]
- t=9-12: A(rem=6)
- t=12-15: D(rem=6) [D arrived at 10]
- t=15-18: B(rem=2)
- t=18-21: C(rem=1)
- t=21-24: A(rem=3)
- t=24-27: D(rem=3)
- t=27-29: B completes (CT=29)
- t=29-30: C completes (CT=30)
- t=30-33: A completes (CT=33)
- t=33-36: D completes (CT=36)

Process	AT	BT	CT	TAT=CT-AT	WT=TAT-BT	Notes
A	0	12	33	33	21	RR Q=3
B	2	8	29	27	19	RR Q=3
C	5	7	30	25	18	RR Q=3
D	10	9	36	26	17	RR Q=3

RR Average TAT: (33+27+25+26)/4 = 111/4 = 27.75 ms
RR Average WT: (21+19+18+17)/4 = 75/4 = 18.75 ms

Q6. What is TSL instruction? Why is it used? Explain the types of semaphore along with the major operations with a simple pseudocode. [1+2+5]

Answer:

TSL (Test and Set Lock) Instruction:

TSL is an atomic (indivisible) hardware instruction that reads the current value of a memory location and sets it to a non-zero value in a single, uninterruptible operation.

```
TSL RX, LOCK    ; Copy LOCK to register RX, then set LOCK = 1
```

Why TSL is Used:

- To implement mutual exclusion in multi-processor systems without OS assistance
- The operation is atomic — no interrupt can occur between the read and set, preventing race conditions
- Used to build spin locks: a process keeps checking LOCK until it becomes 0

TSL-based Mutual Exclusion Pseudocode:

```
enter_critical_section:
    TSL REGISTER, LOCK    ; Test and set
    CMP REGISTER, #0      ; Was it 0?
    JNE enter_critical_section  ; If not, try again (busy wait)
    RET                   ; Enter critical section

leave_critical_section:
    MOVE LOCK, #0         ; Release lock
    RET
```

Semaphore:

A semaphore is a synchronization tool that controls access to shared resources. It is an integer variable that can only be accessed through two atomic operations: wait() (P operation) and signal() (V operation).

Types of Semaphore:

1. Binary Semaphore (Mutex): Can only take values 0 or 1. Used for mutual exclusion. Equivalent to a mutex lock.

2. Counting Semaphore: Can take any non-negative integer value. Used to control access to a pool of resources (e.g., n database connections).

Semaphore Operations Pseudocode:

```
wait(S):            signal(S):
  while S <= 0        S = S + 1
    do nothing;       // wake up a waiting process
  S = S - 1;
```

Binary Semaphore for Mutual Exclusion:

```
Semaphore mutex = 1;

Process Pi:
  wait(mutex);        // acquire lock
  // Critical Section
  signal(mutex);      // release lock
```

Counting Semaphore Example (n resources):

```
Semaphore S = n;    // n available resources

Process:
  wait(S);          // acquire a resource (S--)
  // Use resource
  signal(S);        // release resource (S++)
```

Q7. What information does a Process Control Block contain? Write the difference between thread and process. [5]

Answer:

Process Control Block (PCB):

A PCB (also called Task Control Block) is a data structure maintained by the OS for every process. It stores all information needed to manage a process. When a process is created, the OS creates a PCB for it.

Information stored in PCB:

- Process ID (PID): Unique identifier for the process
- Process State: Current state (new, ready, running, waiting, terminated)
- Program Counter: Address of the next instruction to execute
- CPU Registers: Contents of all CPU registers (accumulator, stack pointer, etc.) saved during context switch
- CPU Scheduling Information: Priority, scheduling queue pointers
- Memory Management Information: Page tables, segment tables, base/limit registers
- Accounting Information: CPU time used, elapsed time, time limits
- I/O Status Information: List of I/O devices allocated, list of open files
- Parent PID: PID of the parent process

Role of PCB in Context Switching:

- When the OS switches the CPU from one process to another, it saves the current process's state into its PCB and loads the next process's state from its PCB

Q8. What is a periodic task? Manage the scheduling of following real time tasks as per Earliest Deadline First (EDF). [2+4] T1: Execution time = 0.9, Period = 2 T2: Execution time = 2.3, Period = 5

Answer:

Periodic Task:

A periodic task is a task that must execute repeatedly at fixed intervals (periods). Each instance of the task has a deadline equal to the start of the next period. Real-time systems (e.g., aircraft control, medical devices) use periodic tasks.

CPU Utilization Check (Schedulability):

- $U = (e1/p1) + (e2/p2) = (0.9/2) + (2.3/5) = 0.45 + 0.46 = 0.91$
- Since $U = 0.91 < 1.0$, the task set is schedulable (EDF can always schedule if $U \leq 1$)

EDF Scheduling (Earliest Deadline First):

At each scheduling decision, the task with the earliest (closest) deadline runs first.

Deadline schedule (period = deadline):

- T1 periods/deadlines: 2, 4, 6, 8, 10...
- T2 periods/deadlines: 5, 10, 15...

- t=0: Both start. T1 deadline=2, T2 deadline=5. T1 runs (earliest deadline).
- t=0.9: T1 completes. T2 runs.
- t=2: T1's new deadline=4. T2 has deadline=5. T2 still running. T1 preempts (deadline 4 < 5)... Actually T2 has already been running for 1.1 units, remaining=1.2.
- t=2: T1 arrives again (deadline=4). T2(deadline=5). T1 runs.
- t=2.9: T1 completes. T2 resumes (remaining=1.2).
- t=4.1: T2 completes. T1 arrives at t=4 (deadline=6). T1 runs.
- t=5: T2 arrives (deadline=10). T1 (deadline=6) continues.
- t=5.0: T1 finishes (started 4.0, exe=0.9, done 4.9). At t=4.9 T1 done. T2 runs.
- This pattern repeats with EDF always picking the task whose deadline is earliest.

Key Properties of EDF:

- EDF is optimal for preemptive scheduling of periodic tasks on a uniprocessor
- Any schedulable task set (U ≤ 1) can be scheduled by EDF
- Unlike Rate Monotonic (RM), EDF can utilize up to 100% of CPU

Q9. Explain Context Switching with an example. [2+8] (with FCFS and scheduling problems)

Answer:

Context Switching:

Context switching is the process of saving the state (context) of a running process into its PCB and loading the state of another process from its PCB, so that the CPU can be given to that other process. It is the mechanism that allows multitasking.

Steps in Context Switch:

1. Save CPU registers (PC, stack pointer, etc.) of current process to its PCB
2. Update process state in PCB (e.g., Running → Ready or Waiting)
3. Move PCB to appropriate queue (ready queue or wait queue)
4. Select next process from ready queue via scheduling algorithm
5. Load CPU registers from PCB of selected process
6. Update process state to Running
7. Jump to the instruction pointed by the restored Program Counter

Context switch is pure overhead — no useful work is done during the switch. Modern OS minimize context switch time using hardware support.